

Course Outline: Representación de objetos en Python

Title: Representación de objetos en Python **Skill:** Skill 32 — Trabajar con datos estructurados de tipo objeto en Python **Learnpack:** + Representación de objetos en Python **File:** s32-w11d33-representacion-de-objetos-en-python **Prerequisite:** Programación Orientada a Objetos (s32-w11d33-programacion-orientada-a-objetos) **Target Audience:** Beginner developers who understand OOP concepts and are ready to write Python class syntax **Total Lessons:** 11 **Format:** Mixed (18% hands-on = 2 lessons)

Course Description

You already know what a class is and why OOP exists. Now it's time to write actual Python. This course takes you from the conceptual model — blueprints, instances, attributes, behavior — to real, working code. You'll define classes, set attributes in constructors, add methods that act on object state, and manipulate collections of objects. By the end, you'll be able to represent any real-world entity as a Python class and use it confidently in your programs.

Course Philosophy

This course emphasizes:

- Syntax as a natural extension of concepts already understood — no re-introduction of what a class is
 - State and behavior as inseparable — a class that has data should also have methods that operate on it
 - Learning through immediate feedback — code that runs and produces output from the first exercise
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Writing Python Classes	4
02 - Using Objects	4
03 - Assessment	1
04 - Outro	1
Total	11

00.0 From Concept to Code

Learning Objectives:

- Understand what this course builds on top of the previous OOP lesson
- Know what topics will be covered and in what order

Content Outline:

- Quick bridge: in the previous lesson you learned what a class is, what an object is, and the four OOP principles — this course starts there
- What this course covers: how Python expresses those concepts in actual syntax
- Section preview: Section 01 (writing classes), Section 02 (using objects and manipulating them)

Transitions: This lesson sets expectations and motivates the course. Section 01 begins immediately after.

Key Principles:

- Conceptual understanding is the map — this course is the road
- Python's class syntax is deliberately readable — it mirrors the concepts you already know

Examples:

- You defined a `Warrior` conceptually as "something with a name and health that can take damage" — this course shows you exactly how to write that in Python
 - By the end of this course, `warrior1 = Warrior("Aragorn", 100)` will be natural and understood line by line
-

01.0 Writing Python Classes

Learning Objectives:

- Understand the overall structure of a Python class definition
- Recognize the role of `__init__` as the constructor
- Understand what `self` is and why it appears as the first parameter in every method

Content Outline:

- The `class` keyword: how Python introduces a new class
- The constructor: `__init__` is a special method called automatically when an object is created
- `self`: a reference to the instance being created or used — it is not a keyword, it is a convention (the first parameter of any instance method, by agreement)
- The anatomy of a minimal class: class name, `__init__`, attribute assignments
- Sub-lesson preview: 01.1 dives into attributes and values; 01.2 adds methods and behavior

Transitions: This lesson gives students the skeleton. 01.1 and 01.2 fill it in.

Key Principles:

- Every instance method must declare `self` as its first parameter — Python passes the instance automatically
- `__init__` is where the object is built — it runs the moment you create an instance
- `self` is not magic: it is just the name the community agreed on. You could write `def __init__(this, name)` and it would work — but never do it

Examples:

```
class Warrior:
    def __init__(self, name):
        self.name = name
```

- `Warrior` is the class name
- `__init__` runs when you write `Warrior("Aragorn")`
- `self.name = name` saves the argument onto the instance

01.1 Attributes and Values

Learning Objectives:

- Distinguish between an attribute (the label) and a value (what it holds)
- Always define instance attributes inside `__init__`
- Understand why each object gets its own copy of every attribute

Content Outline:

- Attribute vs Value: `self.health` is the attribute; `100` is the value stored in it
- Why `__init__` is the right place: defining attributes here guarantees every instance has them from the moment of creation
- Instance independence: two `Warrior` objects share the same class definition but have completely separate attribute values
- Default values: setting a predictable starting state in `__init__`

Transitions: Attributes hold data. 01.2 adds methods — the behavior that acts on that data.

Key Principles:

- An attribute is a variable that belongs to a specific instance, not to the class as a whole
- Defining attributes only in some methods (not in `__init__`) leads to inconsistency — one object may have an attribute another doesn't
- Each instance has its own copy: changing `warrior1.health` does not touch `warrior2.health`

Examples:

```
class Warrior:
    def __init__(self, name, health):
        self.name = name        # attribute: name, value: whatever was passed
```

```
in
    self.health = health # attribute: health, value: whatever was passed
in

warrior1 = Warrior("Aragorn", 100)
warrior2 = Warrior("Legolas", 90)
# warrior1.health is 100, warrior2.health is 90 – completely independent
```

01.2 Adding Behavior with Methods

Learning Objectives:

- Define methods that operate on an instance's own data
- Understand that a well-designed class holds both state and the behavior that changes it
- Recognize the static class anti-pattern and explain why it is not OOP

Content Outline:

- Methods as functions inside a class that always receive `self`
- State + Behavior: a class's methods should act on `self`'s attributes — they are not standalone functions
- Calling a method: `warrior.take_damage(25)` passes the instance as `self` automatically
- Anti-pattern — La Clase Estática: a class whose methods never use `self` is just a namespace for functions, not an object. It provides none of the benefits of OOP (encapsulation, instance independence, reusability)

Transitions: This lesson completes the class definition. 01.3 is the hands-on challenge to write one from scratch.

Key Principles:

- A method that doesn't use `self` doesn't need to be in a class
- State and behavior belong together: if an object tracks health, the object should also know how to change it
- The method signature always starts with `self`, but the caller never passes it explicitly

Examples:

```
class Warrior:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def take_damage(self, amount):
        self.health -= amount # modifies THIS instance's health

warrior = Warrior("Aragorn", 100)
warrior.take_damage(25)
```

```
print(warrior.health) # 75

# Anti-pattern: static class (no self usage – this is just grouped functions,
# not OOP)
class MathUtils:
    def add(x, y): # no self – this is not an instance method
        return x + y
```

01.3 Define a Python Class 🖥️

Challenge Prompt: Define a `Warrior` class with a `name` and `health` attribute set in `__init__`, and a `take_damage(amount)` method that subtracts `amount` from `health`. Create two warrior instances and call `take_damage` on the first one, then print each warrior's name and current health.

Solution Code:

```
class Warrior:
    def __init__(self, name, health):
        self.name = name
        self.health = health

    def take_damage(self, amount):
        self.health -= amount

warrior1 = Warrior("Aragorn", 100)
warrior2 = Warrior("Legolas", 90)

warrior1.take_damage(25)

print(f"{warrior1.name} has {warrior1.health} health")
print(f"{warrior2.name} has {warrior2.health} health")
```

Challenge Code:

```
class Warrior:
    def __init__(self, name, health):
        # set self.name and self.health
        pass

    def take_damage(self, amount):
        # subtract amount from self.health
        pass

warrior1 = Warrior("Aragorn", 100)
warrior2 = Warrior("Legolas", 90)

warrior1.take_damage(25)
```

```
print(f"{warrior1.name} has {warrior1.health} health")
print(f"{warrior2.name} has {warrior2.health} health")
```

02.0 Using Objects

Learning Objectives:

- Understand what instantiation means and what happens when you create an object
- Recognize that each instance is fully independent
- Understand the overall pattern of working with objects after they are created

Content Outline:

- Instantiation: calling the class like a function creates an object — `warrior = Warrior("Aragorn", 100)`
- What happens under the hood: Python creates a new object, calls `__init__`, and returns the instance
- Independence: every instance holds its own attribute values and responds to its own method calls
- Sub-lesson preview: 02.1 covers dot notation and property access; 02.2 covers manipulating objects and building more complex structures

Transitions: Section 01 showed how to write a class. This section shows how to use it.

Key Principles:

- The class is the blueprint; the object is the building
- You can create as many instances as you need — each is separate
- Objects live in variables just like any other value

Examples:

```
warrior1 = Warrior("Aragorn", 100)
warrior2 = Warrior("Legolas", 90)
# Both are Warrior objects, but they are completely separate entities
```

02.1 Dot Notation

Learning Objectives:

- Read and write object attributes using dot notation
- Call methods on an object using dot notation
- Understand why modifying attributes directly from outside a class is a design risk

Content Outline:

- Reading a property: `warrior.name` returns the value of `name` on that instance
- Calling a method: `warrior.take_damage(10)` calls the method and passes the instance as `self`
- Writing a property directly: `warrior.health = 50` sets the attribute from outside the class
- Anti-pattern — Acceso Total: modifying attributes directly from outside bypasses any logic the class has (or might need later). If `warrior.xp += 100` is scattered across your code and you later add a level cap, you must find and update every location. Prefer methods that control how state changes.

Transitions: Dot notation is the primary interface to any object. 02.2 extends this to more complex scenarios.

Key Principles:

- `object.attribute` is the universal syntax for accessing anything on an object
- Methods are also accessed via dot notation — `object.method()` — the difference is the parentheses
- Attribute access from outside is not forbidden, but write methods for any change that has business logic attached

Examples:

```
warrior = Warrior("Aragorn", 100)

print(warrior.name)      # "Aragorn" – reading an attribute
warrior.take_damage(10)  # calling a method
print(warrior.health)    # 90

# Risky: direct write from outside
warrior.health = 9999    # works, but bypasses any health cap the class
                           might enforce
```

02.2 Object Structures

Learning Objectives:

- Modify object state through method calls
- Build objects that contain other objects as attributes
- Manage a list where every element is an object instance

Content Outline:

- Modifying state: calling methods changes the object's attribute values over time
- Objects within objects: a `Car` class whose `self.wheels` attribute is a list of `Wheel` instances — each wheel is a full object with its own attributes
- Lists of objects: `users = []` where each element is a `User` instance — the list is just a standard Python list, but what's inside it are objects

- Accessing nested objects: `car.wheels[0]` gives the first `Wheel`; `car.wheels[0].pressure` gives that wheel's pressure attribute
- Note on complexity: modeling real systems often means objects that contain other objects. Keep hierarchies shallow — a class should represent one concept, not an entire system.

Transitions: This lesson rounds out the practical use of objects. 02.3 is the hands-on challenge.

Key Principles:

- An attribute can hold any Python value — including another object
- A list of objects is just a list — you iterate, filter, and sort it the same way
- Dot chains work as deep as needed: `car.wheels[0].pressure` — but prefer flat structures when possible

Examples:

```
class Wheel:
    def __init__(self, pressure):
        self.pressure = pressure

class Car:
    def __init__(self, brand):
        self.brand = brand
        self.wheels = [Wheel(32), Wheel(32), Wheel(30), Wheel(31)]

car = Car("Toyota")
print(car.wheels[0].pressure)  # 32 – object within object

# List of objects
users = [User("Ana"), User("Luis"), User("Marta")]
for user in users:
    print(user.name)
```

02.3 Manipulate Objects

Challenge Prompt: Create a list containing three `Product` instances (each with a `name` and `price`), then print the name and price of the most expensive one.

Solution Code:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

products = [
    Product("Laptop", 999),
    Product("Mouse", 25),
```



```
        Product("Monitor", 350),
    ]

    most_expensive = max(products, key=lambda p: p.price)
    print(f"{most_expensive.name}: ${most_expensive.price}")
```

Challenge Code:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

# Create a list with three Product instances
products = []

# Print the name and price of the most expensive product
```

03.0 What You Know Now 🧠

Learning Objectives:

- Demonstrate understanding of Python class syntax and instance creation
- Identify correct and incorrect uses of `self`, attributes, methods, and dot notation
- Apply object manipulation concepts to scenario-based questions

Question Format: Multiple choice (8 questions)

Questions:

1. Where should instance attributes always be defined? a) Outside the class, as global variables b) In the `__init__` method ☒ c) In whichever method first uses them d) In a separate `setup()` method
2. What is `self` in a Python class method? a) A Python keyword with special reserved meaning b) A reference to the class itself c) A reference to the current instance of the class ☒ d) A placeholder that Python replaces with the method name
3. You see this code written outside the class: `player.score += 10`. What is the risk? a) It throws a Python `AttributeError` b) It creates a brand-new attribute unrelated to the class c) It bypasses any validation or control logic the class might add later ☒ d) It works fine — Python encourages direct attribute access
4. A class named `MathUtils` has three methods (`add`, `subtract`, `multiply`), none of which use `self`. What is the problem? a) Python requires all methods to use `self` b) This is valid OOP — grouping functions in a class is the point c) This is the static class anti-pattern — it is function grouping, not OOP ☒ d) The methods should use `@staticmethod` and then it is fine

5. You want a `Library` that contains `Book` objects. Which approach correctly models "objects within objects"? a) Store book titles as a list of strings inside `Library` b) Create `Book` instances and store them in a list attribute on `Library` ☒ c) Inherit `Library` from `Book` d) Use a global dictionary with book names as keys
6. You have `car = Car("Toyota")` and `Car` stores four `Wheel` objects in `self.wheels`. How do you access the pressure of the first wheel? a) `car[0].pressure` b) `car.wheels[0].pressure` ☒ c) `car.Wheel.pressure` d) `Wheel(car).pressure`
7. What does calling `Warrior("Aragorn", 100)` do? a) Defines a new class named `Warrior` b) Creates a new object and calls `__init__` with the given arguments ☒ c) Updates an existing `Warrior` instance with new values d) Imports the `Warrior` class from the standard library
8. You have a list `products = [Product("Laptop", 999), Product("Mouse", 25)]`. How do you print the name of the second product? a) `print(products.name[1])` b) `print(Product[1].name)` c) `print(products[1].name)` ☒ d) `print(products[1]["name"])`

Evaluation Criteria:

- 8/8: Full mastery of Python class syntax and object usage
- 6–7/8: Strong understanding with minor gaps
- 4–5/8: Core concepts understood; review sections 01 and 02
- Below 4: Re-read both sections before continuing

Score Interpretation: A score of 6 or above indicates readiness for Skill 33 (ORM), which builds directly on this syntax.

04.0 Outro

Content Outline:

- Course recap: what students accomplished
 - Wrote Python classes from scratch using `class`, `__init__`, and `self`
 - Defined attributes and methods that model real-world entities
 - Created and manipulated object instances using dot notation
 - Built collections of objects and worked with objects inside objects
- Connection to bigger picture: the class syntax you wrote here is the exact same syntax SQLAlchemy uses to define database models in Skill 33 — you are now prepared for that step
- Next steps: Skill 33 (Object-Relational Mapping) takes these Python classes and maps them to database tables
- Resources:
 - Python official docs — Classes: <https://docs.python.org/3/tutorial/classes.html>
 - Real Python — Object-Oriented Programming in Python 3: <https://realpython.com/python3-object-oriented-programming/>
- Encouragement: You can now represent any entity — a user, a product, a game character, a database row — as a Python object. That is a fundamental skill that shows up in every Python codebase you will ever work with.

Note: This is conclusion only — no theory, exercises, or assessment.
